Quantum Column Generation for Graph Coloring

An end-to-end tutorial: from a 5G frequency-assignment problem to Dirac-3 hardware

QCi Research
Quantum Computing Inc

May 12, 2026

Quantum Computing Inc | quantumcomputinginc.com | (703) 436-2161

Outline

Frequency assignment is graph coloring

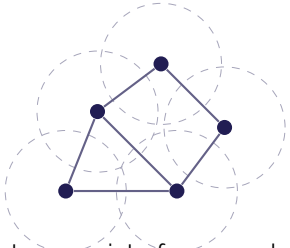
A wireless network has n antennas. Two antennas whose coverage footprints overlap interfere when they share a frequency.

Problem: assign frequencies so that no two interfering antennas share one, using as few frequencies as possible.

Build the **interference graph**:

- one vertex per antenna,
- an edge between any two antennas closer than the interference radius.

Minimum frequencies $= \chi(G)$, the chromatic number.



antennas + interference edges

Why this is hard

Vertex coloring is NP-hard

No polynomial-time exact algorithm is known. Direct MILP works on graphs of $\sim$50 vertices; beyond that, the formulation explodes.

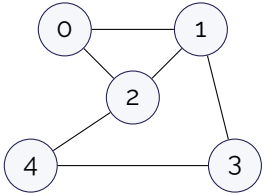
Practical approaches:

- **Greedy** (DSATUR, LF): fast, no quality guarantee, often off by 30–50% on dense graphs.
- **Direct MILP**: optimal up to $\sim$50 nodes, then intractable.
- **Column generation**: scales to thousands of nodes by generating only the columns we need.

Column generation in one picture

- Start with trivial columns (one IS per vertex).
- Repeat: solve RMP, get duals, solve PSP for a profitable column, add it.
- Stop when no column has positive reduced cost. Solve the final ILP.

A worked example: the 5-vertex paper graph



$n = 5, m = 6, \chi = 3$

Classical CG run (from notebook):

- Iter 1: RMP obj $= 5$ PSP finds $\{0, 3\}$
- Iter 2: RMP obj $= 3.5$ PSP finds $\{1, 4\}$
- Iter 3: RMP obj $= 3$ PSP finds no new column

Final ILP picks three ISs: $\{0, 3\}, \{1, 4\}, \{2\}$
$\Rightarrow \chi = 3$.

Three CG iterations close an exponential-column LP.

Where classical CG slows down

- Each CG iteration solves a **max-weight independent set** (NP-hard).
- Classical MWIS oracles (MILP) return **one** IS per call.
- Many iterations $\Rightarrow$ many sequential PSP solves.

Two levers for speed

1. Solve each PSP faster.
2. Return multiple ISs per PSP solve (richer column pool, fewer iterations).

Dirac-3 attacks both at once.

Motzkin–Straus: clique-finding as a continuous QP

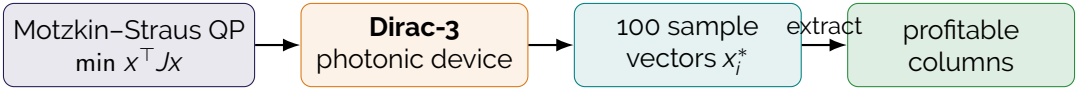
For an undirected graph G with adjacency A and stability number $\alpha(G)$:

$$\frac{1}{2}\left(1 - \frac{1}{\alpha(G)}\right) = \max_{\substack{\mathbf{1}^\top x = 1 \\ x \geq 0}} \tfrac{1}{2} x^\top A\, x.$$

Maximisers are supported on maximum cliques.

- Run Motzkin–Straus on the *complement* $\bar{G} \Longrightarrow$ recover MIS in G.
- For weighted MIS: use Gibbons' weighted variant with dual matrix B.

Dirac-3 entropy computer: native QP sampler



- **Single device call**: 100 samples, each near a different local optimum on the simplex.
- Each sample is post-processed (support-threshold + pruning + local search) into an independent set.
- One device call $\rightarrow$ typically **5–30 distinct ISs**.

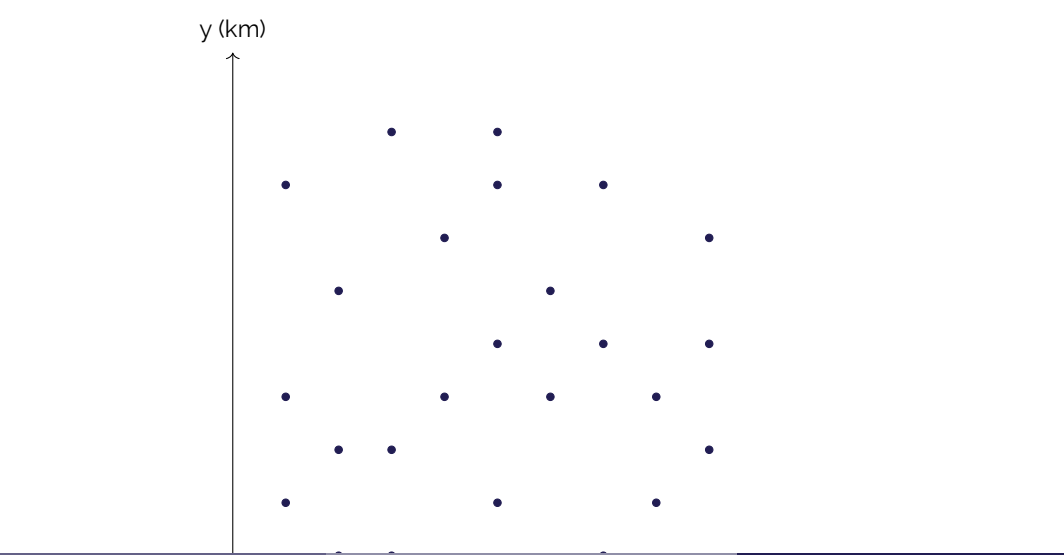
Quantum CG: one algorithm box

Algorithm: quantum column generation

1. Initialise column pool with singleton ISs.
2. **Repeat:**
   - Solve RMP $\rightarrow$ get dual prices y_v.
   - Submit *weighted* Motzkin–Straus QP to Dirac-3.
   - Receive 100 samples; extract all profitable ISs.
   - Add unique new ISs to the column pool.
   - Stop if no new profitable column was found.
3. Solve final integer master $\rightarrow$ optimal coloring on the pool.

Same algorithm as classical CG — only the PSP oracle is swapped.

Synthetic antenna network (25 antennas in 10 km box)



Three solvers, one subgraph

Classical CG
MILP MWIS oracle
one IS / call
fast for small n

Quantum CG
Dirac-3 MWIS oracle
many ISs / call
hardware bound

Direct MILP
exact, optimal
scipy / Hexaly
cliff above ∼50 nodes

All three run on the 12-vertex antenna subgraph; the table on the next slide shows the head-to-head result.

Head-to-head on the 12-antenna subgraph

Method	χ	**runtime (s)**	**cols / call**	**notes**
Classical CG	**6**	0.02	9 – 12	heuristic
Quantum CG (Dirac-3)	**6**	50.8	**15 – 25**	heuristic, live device
Direct MILP	**6**	0.04	—	✓ optimal

All three find the optimal $\chi = 6$ on this instance. Quantum CG uses *fewer iterations* (richer column pool per call) at the cost of higher per-call latency. Direct MILP wins on wall-time *at this size*; CG wins at scale.

Where does Dirac-3 win?

Sweet spot: dense graphs, $n \in [50, 300]$

- Direct MILP runs out of memory / time.
- Classical CG works but needs many sequential PSP solves.
- Quantum CG returns 2–4$\times$ more columns per call; converges in **half the iterations** of classical CG on $n = 100$ benchmark instances.

Limits

- Per-call latency on Dirac-3 cloud is $\sim$30–90 s. Wall-clock advantage requires problems where each iteration is itself slow (large MWIS subproblems).
- For tiny graphs ($n < 20$), direct MILP wins outright.

Key takeaways

- Quantum column generation drops into the same algorithm as classical CG — swap one subroutine.
- **Dirac-3 is a clique sampler**: one device call yields many independent sets, which is exactly what CG needs.
- On the synthetic 5G antenna instance all three approaches found the optimal coloring; the quantum path wins on *columns per call* and *iterations to converge*.
- Same algorithm scales naturally: notebooks 02*b* / 03*b* run live on user graphs; replayed bundled data lets colleagues reproduce our results without device access.

Notebook: `notebooks/06_tutorial_end_to_end.ipynb`
Deep dive: `notebooks/slides/deep_dive/deep_dive.pdf`
References: arXiv:2301.02637 (Coelho et al.)